

Exercise Set for DRL Module 3

Medium-difficulty Python practice on DQN, DDQN, REINFORCE, and Actor-Critic

This document is designed for students who have already attended the lecture and completed the Python tutorial. The aim is not merely to run code, but to connect equations, implementation details, and observed learning behavior.

Purpose of this exercise set

Module 3 moves from value-based deep reinforcement learning to improved target design and then to direct policy learning. The exercises below follow the same path. Students first revisit DQN, then modify it into DDQN, then implement REINFORCE, and finally build a simple actor-critic agent. The set is deliberately *medium* in difficulty: each task requires implementation, experimentation, and interpretation, but each task remains feasible with the tutorial material as a starting point.

Expected learning outcome. After completing this exercise set, students should be able to explain not only *what* each algorithm does, but also *which line of code corresponds to which learning signal*, and *why the observed behavior matches the theory*.

1. Working assumptions and submission format

1.1 Recommended software stack

Students may work with the same stack as in the tutorial:

- Python 3.10 or later
- `gymnasium[classic-control]`
- `torch`
- `numpy`
- `matplotlib`

1.2 Recommended environment choices

To keep the comparison fair and implementation manageable, the following environments are recommended:

- `CartPole-v1` for Exercises 1–4
- Optional extension: `Acrobot-v1` after the main tasks are finished

1.3 What students should submit

For each exercise, students should submit:

1. the Python file or notebook,
2. one short report (about 1 page per exercise),
3. at least one training plot,
4. brief answers to the interpretation questions.

1.4 Minimum experimental discipline

Students should avoid reporting a single lucky run. For each exercise, the recommended minimum is:

- run at least 3 random seeds,
- report episode return curves,
- comment on stability, variance, and sample efficiency,
- state the chosen hyperparameters.

2. Exercise 1: Rebuild and analyze DQN

Exercise goal

Students rebuild the essential DQN training loop and verify the role of *replay*, *target networks*, and the *TD target*. The main objective is to connect the DQN equations to code rather than to produce the strongest possible score.

Task statement

Implement a DQN agent for **CartPole-v1**. The implementation should include:

- an online Q-network $Q_{\theta}(s, a)$,
- a target Q-network $Q_{\theta^{-}}(s, a)$,
- an experience replay buffer,
- epsilon-greedy exploration,
- mini-batch training with the target

$$y_t = r_{t+1} + \gamma(1 - d_t) \max_{a'} Q_{\theta^{-}}(s_{t+1}, a').$$

Students should then compare the full DQN agent against one weakened variant:

- either *without replay* or *without a target network*.

Steps required to solve the exercise

1. Build a small multilayer perceptron that maps the state vector to one Q-value per discrete action.
2. Implement a replay buffer that stores $(s_t, a_t, r_{t+1}, s_{t+1}, d_t)$.
3. Write an epsilon-greedy action-selection function.
4. During training, sample a mini-batch from replay and compute:
 - the predicted Q-values $Q_{\theta}(s_t, a_t)$,
 - the target values y_t using the target network,
 - the loss, for example MSE or Huber loss.
5. Update only the online network with backpropagation.
6. Every C steps, copy parameters from the online network to the target network.
7. Log episode returns and the training loss.
8. Disable one stabilizing component and compare the learning behavior to the full DQN implementation.

Hints

- Students should first verify that the network output has shape `[batch_size, action_dim]`.
- When extracting $Q_\theta(s_t, a_t)$ from the network output, the code should select the component corresponding to the action actually taken.
- A target network should be used only for target computation. Gradients should not flow through it.
- The comparison variant does not need to be strong; it only needs to make the role of the missing component visible.

Common mistakes to avoid

Typical implementation errors are: forgetting to mask terminal states, updating the target network every gradient step by mistake, or computing the target with the online network when the exercise specifically asks for DQN.

What the report should discuss

Students should answer the following:

1. Why does replay help stochastic gradient descent?
2. Why does the target network reduce the moving-target problem?
3. What changed in the learning curves when replay or the target network was removed?

3. Exercise 2: Convert DQN into DDQN and compare them**Exercise goal**

Students modify only the target computation and observe how a seemingly small code change can alter the learning behavior. This exercise targets conceptual precision: DDQN is not a new family of algorithms, but a refinement of DQN.

Task statement

Starting from the DQN code of Exercise 1, implement DDQN by replacing the DQN target with the DDQN target:

$$y_t = r_{t+1} + \gamma(1 - d_t)Q_{\theta^-} \left(s_{t+1}, \arg \max_{a'} Q_\theta(s_{t+1}, a') \right).$$

Students should train both DQN and DDQN with the same environment and similar hyperparameters, then compare the results.

Steps required to solve the exercise

1. Reuse the architecture, replay buffer, optimizer, and target-sync logic from Exercise 1.
2. In the target computation, first use the *online* network to choose the greedy next action.
3. Then use the *target* network to evaluate that chosen action.
4. Keep every other part of the pipeline unchanged.
5. Train both DQN and DDQN across several seeds.

- Plot episode return for both methods on the same figure.
- Additionally, track one diagnostic quantity, for example the average target value or average predicted Q-value during training.

Hints

- Students should be careful not to evaluate the greedy next action with the online network in the final target. That would remove the DDQN separation principle.
- The strongest comparison is obtained when the same hidden sizes, learning rate, epsilon schedule, and replay settings are used for both algorithms.
- If CartPole is solved too quickly to show much difference, students may shorten training, reduce network size, or optionally repeat the comparison on **Acrobot-v1**.

What the report should discuss

Students should answer the following:

- What is the difference between *action selection* and *action evaluation* in DDQN?
- Which line of code changed relative to DQN?
- Did DDQN produce more stable or less optimistic value estimates in the observed runs?
- If the difference was small on CartPole, why might that happen?

4. Exercise 3: Implement REINFORCE and study variance

Exercise goal

Students move from value-based methods to direct policy learning. The exercise emphasizes the meaning of a stochastic policy, the Monte Carlo return G_t , and the high-variance nature of REINFORCE.

Task statement

Implement a REINFORCE agent with a stochastic policy for **CartPole-v1**. The policy network should output action probabilities using a softmax layer. For one full episode, the student implementation should:

- collect the trajectory,
- compute the returns backward,
- form the actor loss

$$L_{\text{actor}} = - \sum_t \log \pi_{\theta}(a_t | s_t) G_t,$$

- update the policy parameters.

Then students should add *return normalization* or a simple *baseline* and compare the behavior.

Steps required to solve the exercise

- Build a policy network that outputs logits for each discrete action.
- Convert logits into probabilities using softmax.
- Sample actions from the resulting categorical distribution.

4. Store, for every time step, the log-probability of the sampled action and the reward obtained afterward.
5. At the end of the episode, compute the Monte Carlo returns G_t backward.
6. Form the policy loss and run one gradient update.
7. Repeat this for many episodes and plot the episode return curve.
8. Add one variance-reduction mechanism:
 - either normalize the returns within each episode,
 - or subtract a simple baseline such as the mean return of the episode or a running average baseline.
9. Compare the original REINFORCE implementation against the modified one.

Hints

- Students should remember that REINFORCE is updated after the full episode, not after every single step.
- The log-probability must correspond to the sampled action, not to the full vector of action probabilities.
- If training is unstable, students should inspect the scale of the returns and the learning rate.

Common mistakes to avoid

A common mistake is to store only the action index but not the corresponding log-probability. Another common mistake is to forget that the reward at time $t + 1$ contributes to the return G_t .

What the report should discuss

Students should answer the following:

1. Why is REINFORCE considered on-policy?
2. Why can the Monte Carlo return create high-variance updates?
3. Did return normalization or the baseline make the learning curve smoother?
4. In this exercise, what does the term $\log \pi_{\theta}(a_t | s_t)$ mean operationally in code?

5. Exercise 4: Build a simple actor-critic agent

Exercise goal

Students implement the bridge between Monte Carlo policy gradients and value-based learning. The critic estimates value; the actor uses a TD-based learning signal instead of the full return.

Task statement

Implement a one-step actor-critic agent for **CartPole-v1**. The solution should include:

- an actor network representing $\pi_{\theta}(a | s)$,

- a critic network representing $V_\phi(s)$,
- a TD target

$$y_t = r_{t+1} + \gamma(1 - d_t)V_\phi(s_{t+1}),$$

- a TD error

$$\delta_t = y_t - V_\phi(s_t),$$

- a critic loss based on squared TD error,
- an actor update weighted by δ_t .

Students should compare actor-critic with the REINFORCE implementation from Exercise 3.

Steps required to solve the exercise

1. Build an actor network that outputs action logits.
2. Build a critic network that outputs a single scalar value estimate.
3. At each environment step, sample an action from the actor.
4. Observe the transition and compute the one-step TD target using the critic.
5. Compute the TD error δ_t .
6. Update the critic to make $V_\phi(s_t)$ closer to the TD target.
7. Update the actor using a loss proportional to $-\log \pi_\theta(a_t | s_t) \delta_t$.
8. Train the agent and compare its learning curve with REINFORCE.

Hints

- Students may use either separate optimizers for actor and critic or one optimizer per network.
- The actor should not backpropagate through the critic target in an uncontrolled way. A detached TD error is usually cleaner for this exercise.
- Compared with REINFORCE, actor-critic usually updates more frequently because it does not need to wait for the full episode return.

What the report should discuss

Students should answer the following:

1. What role does the critic play in the actor update?
2. Why can actor-critic be less noisy than REINFORCE?
3. Did actor-critic learn faster or more stably in the experiments?
4. In practical terms, what does the TD error tell the actor about the selected action?

6. Optional extension: one compact comparative study

Students who finish early may prepare a final comparison table across all implemented methods. A concise study may include:

- DQN,
- DDQN,
- REINFORCE,
- actor-critic.

The comparison table may summarize:

- on-policy vs off-policy,
- whether replay is used,
- whether a target network is used,
- whether the main learning signal is a TD target, a Monte Carlo return, or a TD error,
- observed sample efficiency,
- observed training variance.

7. Recommended report structure for students

A clean report may use the following structure for each exercise:

1. **Objective:** what the exercise is meant to test.
2. **Method:** what was implemented and with which hyperparameters.
3. **Results:** plots, average return, and stability observations.
4. **Interpretation:** connect the observed behavior to the relevant equation.
5. **Difficulties encountered:** bugs, instability, exploration issues, or sensitivity to hyperparameters.

8. General advice before students start coding

Practical sequence. Students should first make the code run correctly on one seed, then verify shapes and losses, then add logging, and only after that begin comparison experiments. In reinforcement learning, a small implementation bug can easily look like a theory problem.

Hints

A useful debugging habit is to print one mini-batch once, inspect tensor dimensions, and verify the meaning of every stored variable. Another useful habit is to begin with short training runs before launching long experiments.

Closing note

These exercises are designed to reinforce the conceptual bridge built in Module 3: from deep value approximation, to improved target estimation, to direct policy learning, and then to actor-critic methods. A strong solution is therefore not only a working script, but also a technically correct explanation of why each learning signal is constructed the way it is.